Issue Prioritization and Duplicate Detection for GitHub Repositories using Machine Learning

***GUIDE NAME*:**

Dr. B. Karthikeyan

Professor / IT Department

Panimalar Engineering College

Chennai,Tamil Nadu,India

karthikeyan.b32@gmail.com

TEAM MEMBERS:

1st Mohamed Aashath.K

B.Tech- Information Technology

Panimalar Engineering College

Chennai,Tamil Nadu,India

aashathpanimalar@gmail.com

2nd Pradeep M

B.Tech-Information Technology

Panimalar Engineering College

Chennai,Tamil Nadu,India

pradeepkrishna140@gmail.com

3rd Karthik.M

B.Tech- Information Technology

Panimalar Engineering College

Chennai,Tamil Nadu,India

karthikmaharajan2005@gmail.com

Abstract - GitHub has emerged as one of the most popular platforms for collaborating on open-source software development, in which developers work together to log issues, feature requests, and other project-related issues. While the size of the project enhances the number of issues exponentially, manual prioritization of these issues is quite time-consuming. Incorrect prioritization of these issues causes inefficient bug fixes, repetitive issues, and lower software development efficiency. To overcome these challenges, this paper proposes an **Automated Issue Prioritization and Duplicate Detection System for GitHub Repositories using Machine Learning.**

The proposed system processes the GitHub issue information, such as titles and descriptions, using Natural Language Processing (NLP) methods. A machine learning classifier has been used for predicting the priority level of the GitHub issues, and the cosine similarity function has been used to find the duplicate GitHub issues. Issues in the GitHub repositories, whether private or public, without any authentication and using OAuth in GitHub, have been supported by the proposed solution. The suggested solution will enable the developers to prioritize their GitHub issues and remove the necessity of manually searching for the duplicate GitHub issues.

Keywords - GitHub Issues, Machine Learning, Natural Language Processing, Issue Prioritization, Duplicate Detection, Automated Triage

I. INTRODUCTION

Open-source software development has gained popularity due to collaborative spaces such as GitHub, where developers from across the world get to contribute, report issues, and update features. The GitHub platform hosts an issue report system where users get to raise issues or bugs, report improvements, and log tasks related to a specific repository of software. Popular repositories of such software generate an extremely large number of issues, making it hard for developers to resolve using manual analysis.

Prioritization of the issues is also important in the maintenance of the software because the programmer would be required to analyze the issues that require an immediate solution and those requiring solutions later. In most cases, the issues are reviewed manually, and the programmer is required to allocate the issues based on his judgment. The task is both time-wasting and error-prone, particularly when the code bases are extensive. There is also the problem of repeat issues in the system because users often present similar issues.

Recent developments in the fields of Machine Learning and Natural Language Processing offer efficient ways to process text data, including issue names and descriptions. With the help of these, it becomes possible to automatically analyze important aspects from the text, and priorities can be predicted. In addition, a correlation approach may be employed for the detection of similar issues in the text form of problems.

In this scenario, this paper proposes an automatic GitHub issue prioritization and duplicate detection approach via machine learning methods. The proposed approach preprocesses issues as texts and represents them in a quantitative form via TF-IDF. A classification technique is used as part of this approach for prioritizing issues. Moreover, this approach utilizes the Cosine Similarity method for detecting duplicate issues based on texts. This proposed approach will be able to handle public as well as private GitHub repos. The proposed method has been designed to offer an efficient and practical technique for handling issues in real-world software. This method will help increase software system issues efficiency and enable software developers to produce high-quality software systems.

II. RELATED WORK

GitHub has turned out to be one of the leading platforms for open-source software development. This has thereby contributed to the growth in the number of issues such as bugs, requests, and improvements at an unprecedented rate. Handling such issues manually is a difficult task. Therefore, numerous studies have been conducted on automated approaches to handle such issues.

There are some existing methods that focus on issue prioritization based on labels, where issues are labeled as bug, enhancement, or priority, and then the importance of the issues is calculated based on these labels. But this method has limitations because labels are manually assigned, and many issues are left without labels, so the accuracy of this method is compromised. There are some methods where machine learning has been used to prioritize issues into groups of priority based on past experience.

Natural Language Processing techniques such as text preprocessing and TF-IDF vectorization are general approaches that have been employed widely for the analysis of issue descriptions. Classifiers such as Naive Bayes and Support Vector Machines have showed effectiveness in text-based issue classification tasks. However, most of the available systems address only priority prediction without addressing duplicate issue detection, which causes the same issues to be reported multiple times, thereby increasing the effort of maintenance.

Some research has suggested similarity-based approaches using cosine similarity and vector representation of text for the identification of duplicate issues. Although these methods reduce redundancy, they are usually designed in an isolated manner without integrating into a comprehensive issue prioritization system.

In contrast to existing works, the proposed system integrates issue prioritization and duplicate detection into a single automated framework. By combining NLP techniques, machine learning-based priority classification and similarity-based duplicate detection, the system offers a practical and efficient solution for managing GitHub issues. The proposed approach supports both public and private repositories and is also designed as an end-to-end system suitable for the real-world software development environment.

III. SYSTEM DESIGN & METHODOLOGY

This part outlines the architecture, workflow, and approach of the proposed Automated Issue Prioritization and Duplicate Detection in the GitHub Repositories by Machine Learning. The system can be described as a system that uses machine learning and natural language processing in the automatic analysis of the issues in the GitHub system and the identification of the priorities and duplicates.

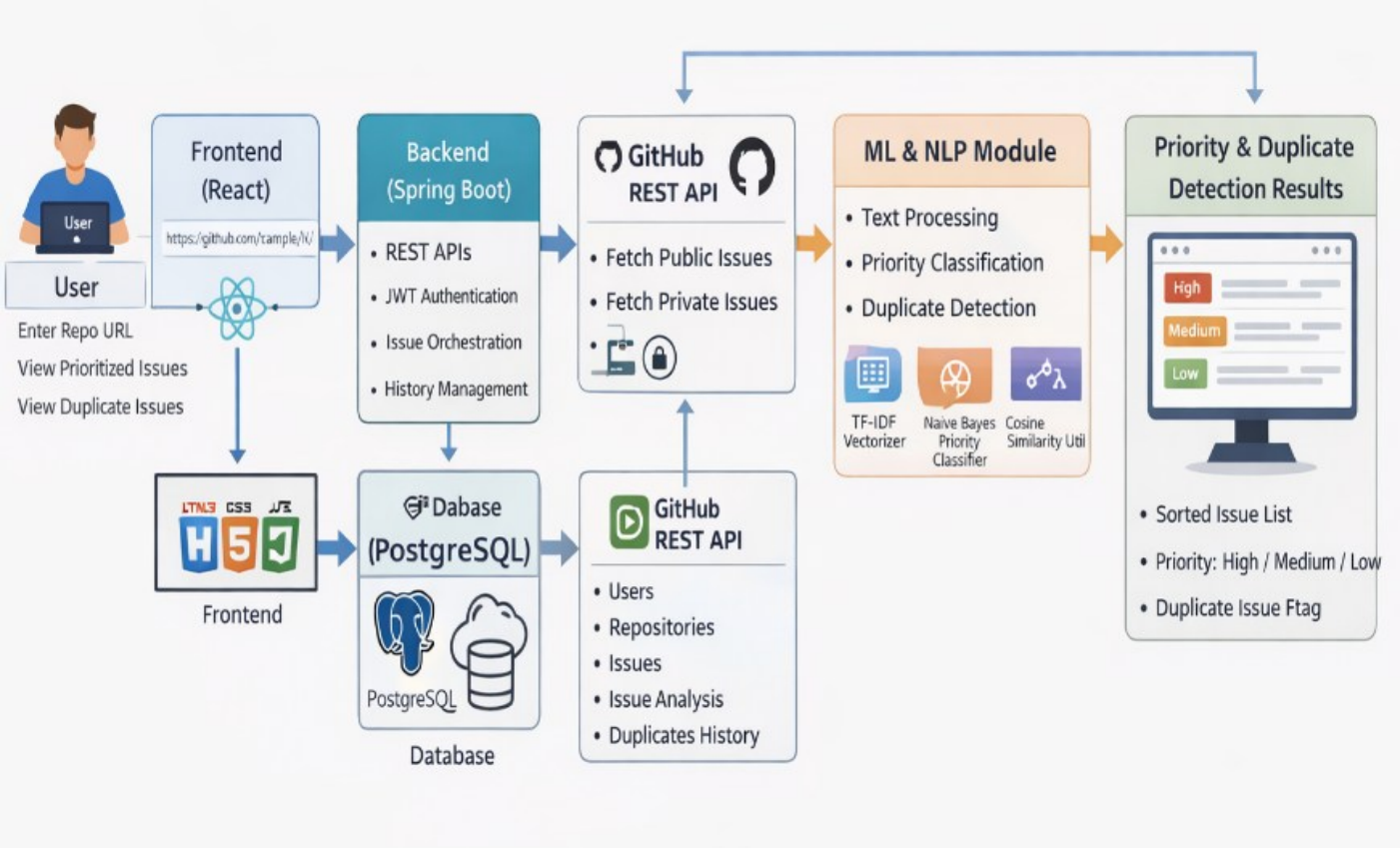
A. System Architecture

The proposed system will comprise five major components namely User Interface, Backend Services, Machine Learning Module, GitHub Integration, and Database Layer in a modular scalable design.

In this regard, the relationship between the user and the system is facilitated by the interface, which enables access to the repository.

- The backend service is used as a central controller for performing many tasks like handling API calls for issues, user authentication, loading issues, and interacting with the machine learning module. The issues from the GitHub repository are obtained from the GitHub REST API. Anyone can access public repos through unauthenticated calls. Authenticated calls made using GitHub OAuth must be done.

The module of machine learning recognizes the extracted issues from the database pertaining to the copy and priority prediction issues. The results are stored in the database and displayed to the person using the system. The system is multi-layered to offer greater results regarding maintainability and security. The system can be quite easily updated in the future.



B. Data Collection and GitHub Integration

This data is collected from the GitHub repositories through the GitHub REST API. The data collected from the issues includes:

- Issue Title
- Issue description (body
- Author details
- Established date and status

In the public repositories, issue information is retrieved without any need to authenticate users. In the private repositories, users get to authenticate their GitHub OAuth in order to securely access issues in their respective repositories.

C. Text Preprocessing (NLP)

Before the application of machine learning algorithms, it is necessary to preprocess the textual information in the issues. This is done by the application of Natural Language Processing techniques to the issue texts by the proposed system.

The preprocessing techniques involved are:

- Lowercase to text conversion
- Removing special characters and punctuation
- Processing text into words through tokenization
- Removing stop words
- Text Normalization

Both the title of the issue and its description are merged and preprocessed to create a clean textual representation of the issue. This is used as an input for the next stage of feature extraction.

D. Feature Extraction using TF-IDF

It can be noticed that the text data obtained in the pre-processing step is converted to Feature Vector using the Term Frequency-Inverse Document Frequency technique. TF-IDF technique makes it easier to identify the key terms in the issue, which remain less affected by more frequently appearing terms.

Each issue is represented by a TF-IDF vector that is informed by the weight of the co-occurrence words for the description of the issue. This is of help in the application of the TF-IDF as the feature vectors for the two tasks of priority classification and duplicate detection. The choice of the TF-IDF is informed by the simplicity and efficiency of the method and is suitable for the process of text classification.

E. Issue Priority Classification

Classification techniques applied through machine learning are applied in determining the priority of the issues on GitHub based on the predicted priority. For example, the priority levels such as High, Medium, and Low may be predetermined and then classified based on the system.

The classification model shall then go ahead and interpret these TF-IDF scores to make predictions using the knowledge that has been gathered on the extent of the issues. This predictive learning of the developed model shall then aid in automating the importance levels of the issues that have been extracted, therefore enabling the developers to have their priorities in regard to the more important issues while running their repositories in an efficient manner.

F. Duplicate Issue Detection

The issue that a large GitHub repository may have duplicate issues. The solution to this issue will be a duplicate issue detection method implemented in a cosine similarity-based system.

The new issue TF-IDF vectors are then contrasted with existing issues. The cosine similarity is calculated to measure the textual similarity between issues. Once the value of similarity exceeds a specific threshold value, then the value is termed as a potential duplicate issue. Not only does it stop duplicate reporting, but it also proves to be an efficient approach as it stops the developers from doing repetitive work.

G. Result Storage and Presentation

The result of the priority levels and the duplication process is stored in a database for future use. The system has a history of the repositories and issues that have been analyzed, and it is therefore possible to trace the history of the analyses. The output is provided in a formatted way, giving the priority levels and the duplicates.

ALGORITHM :

1. **GitHub Repository Authentication Algorithm**
   For authentication of user accounts, and also for making use of OAuth and JWT to connect to their own repositories on GitHub.
2. **JWT Token-Based Authentication Algorithm**
   Manages the sessions between the front and back ends.
3. **Retrieve issues on GitHub Algorithm**
   Fetches data for issues such as title, description, labels, and metadata from GitHub repositories through the REST API offered by GitHub.
4. **Text Preprocessing Algorithm**
   This normalizes problem text by removing stop words and special characters.
5. **TF-IDF Feature Extraction**
   There may be involvement in converting texts of prepared issues into vectors for machine learning.
6. **Issue Priority Classification Algorithm**
   It applies Naïve Bayes, a machine learning algorithm that will categorize the problem under one of the three categories of priority: High, Medium, and Low.
7. **Duplicate Bug Detection Algorithm**
   On the basis of similarities in texts among issues, the duplicate issues are identified by using cosine similarity.
8. **Algorithm for Sorting Issues Based on Priority**
   This is done for sorting the issues based on the estimated priority.
9. **Problem Discussion of the Storage**
   They hold information about priorities and duplication results within the database.
10. **Algorithm for the Presentation of Results**
    The web interface displays concerns labeled as 'prioritized' and 'duplicates' as well.

IV. RESULTS AND DISCUSSION

A. System Output and Performance

The proposed system has been tested on the GitHub repositories with various issues that have different complexities. The system was able to fetch data regarding the issues from GitHub repositories. The system has used natural language processing to process the title and description of the issue. The machine learning model could identify whether an issue should have high, medium, or low priority. In this case, duplication issue detection proved useful in the use of cosine similarity for textual similarity. This was important in the elimination of redundant issue reporting and pointing to previously reported issues by developers.

B. Priority Prediction Analysis

This analysis The results obtained from the task of priority classification show quite clearly that the system is able to assign appropriate priorities to the issues existing in the GitHub repository on its own. Critical issues associated with bugs and failures were assigned higher priorities, while issues in terms of feature requests and minor changes were assigned lower priorities.

C. Duplicate Issue Detection Analysis

The duplicate issue detection algorithm is effective for detecting issues with similar descriptions and titles. Marking duplicate issues is useful in eliminating the issue of duplication of effort and increasing the efficiency of issue handling. The developers are now able to see the previous issues and are not required to deal with a similar issue again and again.

D. Overall System Effectiveness

Estimated Cost of Ownership The combination of issue priority and duplicate issues in the system helps to enhance the efficiency of issue management in GitHub. The system reduces efforts and increases accuracy in issue management. The system allows public and private repositories. The issue management system in the project has a modular structure that can be easily scaled up.

V. CONCLUSION

This paper introduced an Automated Issue Prioritization and Duplicate Detection System for GitHub Repositories with the aid of Machine Learning. This system not only resolves the issue of prioritizing bugs by humans on a massive GitHub repository but also automates the process of issue data analysis, thereby assigning levels of priorities. This is achieved by using NLP techniques, which include text preprocessing and TF-IDF.

The algorithm for machine learning designed above is used for categorizing the issues classified based on the priority, with the categories being High, Medium, and Low. This will enable the developer to prioritize the tasks and take necessary actions with respect to the critical issues. Along with this, the duplication issue system identifies similar issues based on the cosine similarities. This system is used for both private and public repositories.

Specifically, the proposed solution enhances the efficiency of issue management, minimizes the need for manual labor, and also helps developers in preserving the quality of the software. The system is applicable in a real-world setting in the field of software development due to its modularity and application of machine learning.

VI. FUTURE WORK

Although the proposed system is an efficient way of automatically prioritizing issues, as well as identifying if they are duplicates, there are other ways that the proposed system might be enhanced. More complex methods involving deep learning, such as BERT, can be integrated in order to further improve text classification. Another enhancement that can be made for this proposed system is automatic assignment for developers.

Future development of such a system may include tracking issues in real time using a GitHub web hook system, notifications related to high priority issues to alert the developer of such an issue, and also analyzing changes made in a pull request to boost the functionality of such a system.

REFERENCES

1. Yingying He, Wenhua Yang, Minxue Pan, Yasir Hussain, Yu Zhou, "Understanding and Enhancing Issue Prioritization in GitHub," *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.
2. Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, Sebastiano Panichella, "Ticket Tagger: Machine Learning Driven Issue Classification," *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2019.
3. Waleed Alhindi, Abdulrahman Aleid, Ilyes Jenhani, Mohamed Wiem Mkaouer, "Issue-Labeler: an ALBERT-based Jira Plugin for Issue Classification," *IEEE/ACM MOBILESoft*, 2023.
4. Vitor Mesaque Alves De Lima, Jacson Rodrigues Barbosa, Ricardo Marcodes Marcacini, IEEE International Conference on Software Maintenance and Evolution (ICSME), 2024.
5. Saad Shafiq, Atif Mashkoor, Christoph Mayr-Dorn, Alexander Egyed, "NLP4IP: Natural Language Processing-based Recommendation Approach for Issues Prioritization," *IEEE SEAA*, 2021.
6. Moghis Fereidouni, Adib Mosharrof, Umar Farooq, A.B. Siddique, "Proactive Prioritization of App Issues via Contrastive Learning," *IEEE Big Data Conference*, 2022.
7. Panagiota Chatzipetrou, Lefteris Angelis, Per Rovegård, Claes Wohlin, "Prioritization of Issues and Requirements by Cumulative Voting," *IEEE EUROMICRO Conference*, 2010.
8. Juan Carlos B. Somohano-Murrieta et al., "Requirements Prioritization Techniques in the Last Decade," *IEEE CONISOFT*, 2020.
9. Noshin Tasneem, Hazura Zulzalil, Sa'Adah Hassan, "Enhancing Agile Software Development: A Systematic Literature Review of Requirement Prioritization," *IEEE Access*, 2025.
10. Jingjing Liang, Sebastian Elbaum, Gregg Rothermel, "Redefining Prioritization: Continuous Prioritization for Continuous Integration," *IEEE/ACM ICSE*, 2018.

11. Vali Tawosi, Afnan Al-Subaihin, Rebecca Moussa, Federica Sarro, "A Versatile Dataset of Agile Open Source Software Projects," *IEEE/ACM MSR*, 2022.
12. Muhammad Imran Babar, Muhammad Ramzan, Shahbaz A.K. Ghayyur, "Challenges and Future Trends in Software Requirements Prioritization," *IEEE International Conference on Computer Networks and IT*, 2011.
13. Falak Sher, Dayang N.A. Jawawi, Radziah Mohamad, Muhammad Imran Babar, "Requirements Prioritization Techniques and Different Aspects," *IEEE MySEC*, 2014.
14. G. Salton and C. Buckley, "Term-Weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, 1988.
15. T. M. Mitchell, "Machine Learning," McGraw-Hill, 1997.